NCTU-EE IC LAB – Fall 2018

Midterm Exam Reference Answer

1.

(1) In module test(), since variable "out" is connected to a module output, it should be declared as "wire" instead of "reg."

(2) In module test(), when instantiating a mux2_1 module, a name should be given, such as mux2_1 m1(.out(out),…);

(3) In module mux2_1(…), all the module inputs cannot be declared as "reg", it should be declared as "wire."

(4) In module mux2_1(…), the variable "out" is written in the LHS (left hand side) in an always block, thus it should be declared as "reg."

(5) The variable "2_1" violates the naming rule, it should begin with _ or alphabets, such as "w2_1."

2.

(a) Since the combinational logic uses blocking assignment, which has the order from top to down. Thus if "a" signal changes, y=x|c|d will be performed first, where c and d do not change now. Then x = a&b, and all the tasks done. y finally uses the old value of x, which is from the previous "a" & b.

(b) always @(*) will put all the variables in the RHS (right hand side) or those in if / case blocks in the sensitive list. If always @(*) is written in this case, x will be also put in the sensitive list. Thus continue after (a), After x=a&b, the sensitive list will be triggered again, and y will finally equals to x|c|d, where x is the updated "a" & b.

(c) There are various ways to make it right, here provides two examples:

```
always@(a or b or c or d) begin
    x = a & b;
    y = x | c | d;
end
always@(a or b or c or d) begin
    y = (a&b) | c | d;
    x = a & b;
end
```

Since "|" operation has the higher priority than "&." If you write the 2nd answer, you should add brackets "()" to a&b. If not, you will get 1 deducted points.

3.

(a) Advantages: Since for those cells we are commonly used in digital circuit like AND gate, OR gate, have already been designed in circuit level and even have already been layout. Thus we only need to describe the design behavior, then the synthesis and P&R(place and route) will make the following steps simpler.
Disadvantage: The cell-based flow costs less time to perform, the designer can only designed from gate level or rtl level, which cells are already been designed. If the designer wishes to design a customized design which particularly optimizes an index like area or power, the cell-based flow cannot give an optimized solution.
Supplement: Another important advantage is that cell-based digital design flow is very easy to do the "technology scaling." That is, if I change the technology process from umc 0.18um to umc 90nm, it takes very little time to do since you can use the same behavior netlist as before. You just have to switch the technology file, and most progress as same as before. (This is not mentioned in slides so you will not get deducted point not writing this)

(b) RTL simulation: Just simply provides the simulation netlist (.v files). (These .v files may include your design, pattern, designware or macro behavior netlist) It will gives the simulation result such as waveform (.fsdb) and text information (with $display or $fwrite function in pattern)
Synthesis: Provide the netlist file you are going to synthesize (.v), the library corresponding to the technology process (.db or .lib), which provides the internal delay / power / capacitance / setup time, etc., various information of the cell. It will generate the synthesized netlist (.v), the delay description file of the netlist in standard delay format (.sdf). The report may also be generated to see if there is Latch (syn.log), if the slack is met (.timing) and the resource and area (.resource, .area).
Gate level simulation: Provide the synthesized netlist (.v) and the delay description file (.sdf) generated from synthesis. Also the standard cell netlist description (like umc18_neg.v) Pattern may also be provided and for memory you have to include the behavior netlist again. The generated file formats are the same as those in RTL simulation.

4.

	`timescale Unit/Precision	**Delay Declaration**	**Actual Delay**
(a)	`timescale 10ns/1ns	#5	50
(b)	`timescale 10ns/1ns	#5.738	57
(c)	`timescale 10ns/10ns	#5.1	50
(d)	`timescale 10ns/100ps	#5.732	57.3

If you write the ceiling of the values (ex: 57->58) you will get 1 deducted points.

5. If the $sdf_annotate() task is not written, the simulation will not read the .sdf file. Then all the delays of the standard cell gates will use the default one (which is usually 1ns, larger than those in the .sdf file). Thus all the path delay becomes longer, and the value will be easily changed in the setup check region, causing the simulation to encounter "Timing violation" warning.
   (If you write the gate will have no delay instead of they will use default delay, you will get 2 deducted points)

6.
   (a) Since our design may be connected to other designs. Take output for example, if both our design use the same clock, and the design I which my output is connected to takes some time to calculate before entering a Flip-Flop, then the time I can perform the calculation will be less than 1 cycle period. The time I reserve for other designs which can perform their calculations is thus called input / output delay. If the input / output delay is larger, that means I reserve more time for the other designs. For myself, the time which can be used is smaller and the constraint will be stricter.

   (b) reg to reg: $\mathbf{t_{cq}+ t_d+t_s< T_{cycle}}$
   input to reg: $\mathbf{t_i+ t_d+t_s< T_{cycle}}$
   reg to output: $\mathbf{t_{cq}+ t_d+t_o< T_{cycle}}$
   For reg to reg, the delay time from register to another is $\mathbf{t_{cq}+ t_d}$, this time is called data arrival time in the design compiler. It should arrive at the register before the setup time checking point, which is $\mathbf{T_{cycle}- t_s}$, which is called data required time. Thus the equation $\mathbf{t_{cq}+ t_d+t_s< T_{cycle}}$ will be got. For input to reg, the starting point is the module input instead of a flip-flop, thus there is no $\mathbf{t_{cq}}$. The rest of the part will be the same ($\mathbf{t_{cq}+t_d}$ in the other design is $\mathbf{t_i}$, note that both the other and our design may both have combinational delays. On the other hand, in reg to output, the end point is the module output instead of the flip-flop, thus no $\mathbf{t_s}$ will be given. If you do not include combinational delay $\mathbf{t_d}$ in input to reg or reg to output, you will get 1 deducted point for each.

7. As mentioned in Answer 6. The output delay is the time we reserve for the design we are connecting to. The time required for the circuit 2 is $T_{\text{combinational Circuit 2}}+ T_{setup}$, thus the output delay should be at least set to 20+5 = 25ns. (Of course you can set more to reserve more time for the other, but your constraint will get stricter.) Here you have to provide the minimum output delay, which is 25ns.

8. *//synopsys translate_off* simply means do not read the following lines in synthesis, and *//synopsys translate_on* simply means read following line in synthesis. In other words, the line *`include "xxx.v"* will not be read. From rtl simulation view, those two lines are just like comments, and the `include line will still be read. From synthesis view, the line will not be read and the file will not be included. However, this does not mean "xxx.v" has been already synthesis or "xxx.v" does not require to be synthesized. (You will get deducted 1 point if you write these reasons) As mentioned above, it just does not read the "`include …" line.

   Then how does it recognize and treat the module which is included? It depends on case. Here take the examples we have used in labs.

   Designware: The .v file is just a behavior netlist that it cannot be synthesized, thus the non-synthesizable netlist should not be include. The design compiler can recognize the designware module we call with the file "dw_foundation.sldb," which is similar to the technology library slow.db. It gives the information how designware we called can be implemented, such as rpl or cla of the adder. It will synthesized the designware into gate level depends on the constraint such as timing / area we set during synthesis. Thus for designware, the design compiler will still synthesize the module into gate level.

   Memory: The .v file is also a behavior netlist which cannot be synthesized. The synthesis tool can still recognize the memory module from the library file "RA1SH.db," it gives information such as delay, timing constraint, area, capacitance of ports of the memories, that enable the synthesizer to perform static timing analysis (STA) based on these data. However unlike designware, the memory will not be synthesized out. Since memory is a hard macro, a compacted layout architecture which is not built from the standard cell gate. Only the timing / area and other information are used for synthesis.

   Soft IP: The .v file is actually synthesizable. Thus this part is just trying to simulate, including the soft IP in synthesis step instead of in RTL step. The IP will be read as well as the main rtl code. Thus even the "`include …" line is not read, the IP can still be recognized since it is read in design compiler. There is no .db or .sldb files like above to describe it, however itself is just a synthesizable netlist, the synthesis tool can definitely synthesize it out.

9. Soft IP: Synthesizable netlist, which is adapted to several process or even FPGA.
   Firm IP: Netlist which has been synthesized, harder to perform technology scaling but saves time from synthesis.
   Hard IP: The file in layout format, basically it cannot perform technology scaling. It is normally used in P & R progress, but the behavior model may also be given.

10. The memory will not be synthesized, so the behavior netlist that cannot be synthesized is used again in gate level simulation (and it will also be used in post layout simulation). This answer can refer to that in Answer 8, which talks about what synthesizer does to memory. In gate level or post layout simulation, the synthesis or P&R tool have already consider the delay time / capacitance loading of the memory (by the RA1SH.db file), thus the synthesized peripheral circuits (not memory) are able cooperate with the memory without any timing violation. The .db file is a library file like .lib file, providing the information to the synthesis tool, so .db file has no direct relation to the gate level simulation (not required input file). The gate level simulation only uses the non-synthesizable memory netlist.

11. (a) don't touch (b) uniquify (c) ungroup
    For (a), two instantiated modules (add1, add2) call the same module (adder_1b), thus this one cannot be uniquify (for uniquify, all module name should be different.)
    For (b), continued from above, each instantiated module corresponds to only one module. Even if those have the same circuits in the module, the module have different names. Thus this one should be uniquify. (If you write uniquify & don't touch for this one, you will also get correct)
    For (c), some of the circuits like DFF and combinational logic are extracted from the module adder_1b, and the combinational circuits are merged. Thus clearly this one is using the ungroup method.